Drone Mapper — High Level Design Document

This project implements a software simulator for an autonomous drone mapper.

The simulator loads three input files:

- drone_config.txt
- mission_config.txt
- map_input.txt

Then it initializes the drone, mock sensors, mock movement driver, and mapping data structure.
The drone explores the 3D map using deterministic movement and scanning logic, writes the discovered map into:

- map_output.txt

Finally, the simulator compares map_output.txt with map_input.txt and prints a mapping score between 0 and 100.

Explanations of your design considerations and alternatives:

The goal of our project is to simulate a drone that maps a 3D building. The drone itself should not know the real map of the building. It should only learn about the environment by using its sensors and movement commands. Because of that, we designed the project as a simulator around the drone, where the simulator controls the environment and the drone behaves like an independent component. This matches the assignment flow, where the program loads the input files, initializes the drone with mock sensors and drivers, runs a main loop of drone commands, and finally writes the output map and calculates the score.

The *simulator* is responsible for the general flow of the program: reading the configuration files, loading the input map, creating the drone and the mock components, running the mapping loop, writing map_output.txt, and calculating the final score.

The *Drone* class is responsible for the mapping algorithm itself. It decides what command to do next: scan, move forward, rotate, elevate, get location, or finish. The drone does not directly access the real building map, and it only knows what it discovered by scanning.

The real building map is stored in *GroundTruthMap*. This map is loaded from map_input.txt.

The map that the drone builds during the mission is stored in *SparseBuildingMap*. This is the output map that later becomes map_output.txt. At the beginning, cells are unknown. During the mission, the drone updates this map according to the scan results and its movement.

The current position and angle of the drone are stored in *DroneState*. This state is shared by the mock components. The position sensor reads from it, the movement driver updates it after a successful movement, and the lidar sensor uses it to know from where the scan starts.

The configuration files are handled by *ConfigParser*. It reads drone_config.txt and mission_config.txt and creates structured objects such as *DroneConfig* and *MissionConfig*.

We also use strong types for units, such as centimeters and degrees as assignment requires.

An alternative was to put most of the logic directly inside the Simulator class. This would be simpler at the beginning, but would make the simulator large and harder to understand. It

would also mix the drone algorithm with the simulation environment, so we decided to separate them.

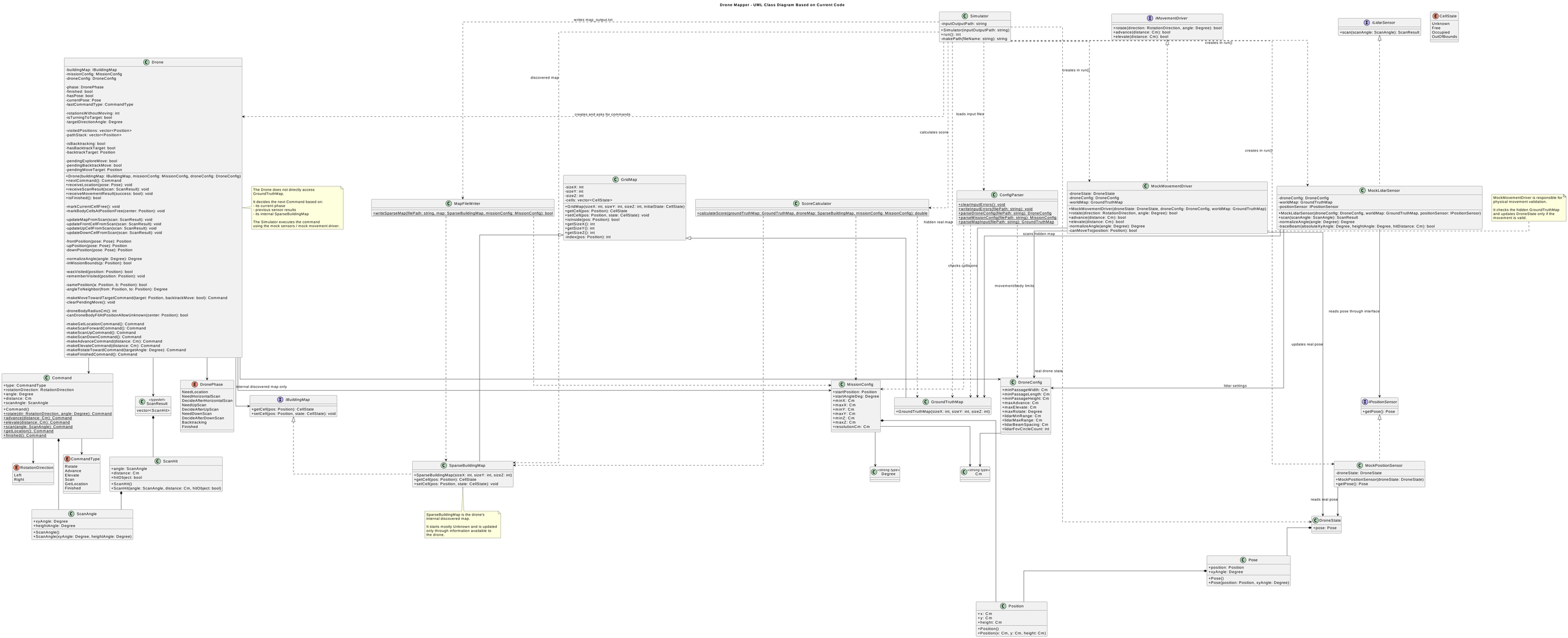
Another design was giving the Drone direct references to the position sensor, lidar sensor, and movement driver. This would have made the Drone more similar to a real physical drone that directly communicates with its hardware components.
We decided not to use this design in the current implementation. Instead, our Drone returns a Command object from nextCommand(), and the Simulator is responsible for executing that command using the mock components. We made this decision for separating the project responsibilities, we think the Drone should focus on the mapping algorithm and on deciding what command should be performed next and the Simulator should manage the simulation environment, connect the Drone to the mock components, and control the main loop.

Our testing approach:

Our testing approach was based on running the simulator with several different input maps and configuration files, and checking that the behaviour matched the expected behaviour according to the assignment requirements.
We tested different map scenarios, including simple open maps, maps with obstacles, boundary cases, and cases where the drone should not be able to move because of an obstacle or because the movement would leave the allowed mission area. In these tests, we expected the movement driver to block invalid movement and keep the drone in a valid position.

In addition to checking the text output files, we used the visualization tool to inspect the drone route visually. The visualization helped us confirm that the drone path, start position, final position, obstacles, and scan behaviour matched what we expected from the input map.

Drone Mapper - UML Class Diagram Based on Current Code



Drone Mapper - Main Flow Sequence Diagram